Naive Bayes Classifier

Abishek

12. November 2024

1. Introduction
2. Python Implementation of Naive Bayes algorithm
3. Pros and Cons
4. References

Introduction

What is Naïve Bayes classifier

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems.
- It is mainly used in text classification that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.

Why is it called Naïve Bayes?

Naïve: Assumes **feature independence**; each feature independently contributes to classification

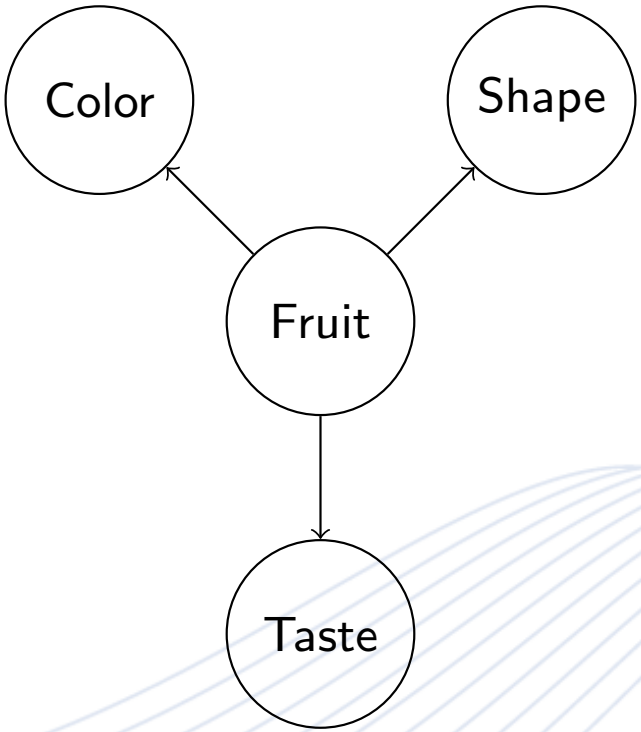

Abbildung: Naïve Bayes Classifier Example

Bayes Theorem

Bayes' Theorem: Also known as **Bayes' Rule** or **Bayes' Law**, it is used to determine the probability of a hypothesis based on prior knowledge and conditional probability.
The formula for Bayes' theorem is given by:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$: **Posterior Probability** - The probability of the hypothesis A given the observed event B.
- $P(B|A)$: **Likelihood Probability** - The probability of the evidence B given that hypothesis A is true.
- $P(A)$: **Prior Probability** - The initial probability of the hypothesis A before observing any evidence.
- $P(B)$: **Marginal Probability** - The probability of observing the evidence B.

Naïve Bayes Classification Process

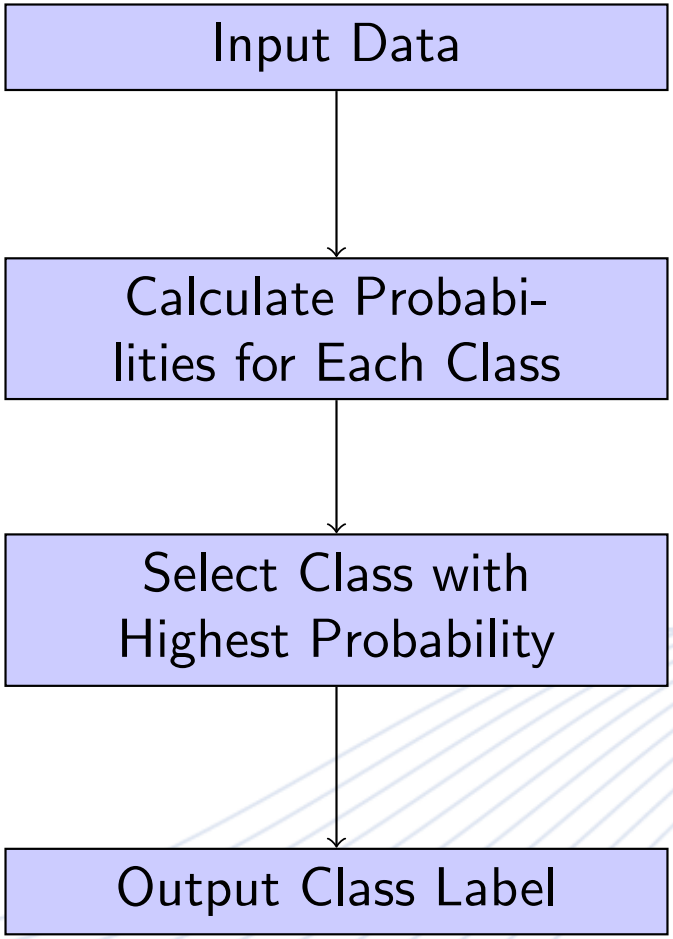

Abbildung: Naïve Bayes Classification Process

Naive Bayes Classification Process: Spam Detection Example

- **Step1: Input Data** Given a new email: "Win a prize now!".
- **Step2:calculate probabilities for each class (Spam vs. Not Spam) using Bayes' theorem:**

$$P(\text{Spam} \mid \text{Email}) = \frac{P(\text{Email} \mid \text{Spam}) \cdot P(\text{Spam})}{P(\text{Email})}$$

where:

- **Likelihood**: $P(\text{Email} \mid \text{Spam})$ represents the probability of observing words like "Win,Prize,änd Nowïn spam emails.
- **Prior**: $P(\text{Spam})$ is the proportion of past emails that were spam.
- **Normalization**: $P(\text{Email})$ ensures that probabilities sum to 1.

- **Step 3: Select the Class with the Highest Probability**
  Compare $P(\text{Spam} \mid \text{Email})$ and $P(\text{Not Spam} \mid \text{Email})$. The class with the higher probability is chosen as the predicted class.

- **Step 4: Output Class Label**
  The result is that the email is classified as Spam if $P(\text{Spam} \mid \text{Email})$ is higher than $P(\text{Not Spam} \mid \text{Email})$.

When to use the Naïve Bayes Classifier algorithm:

- If you have a moderate or large training data set.
- If the instances have several attributes.
- Given the classification parameter, attributes which describe the instances should be conditionally independent.

Types of Naive Bayes classifiers

Type	Data Type	Example Use
Gaussian	Continuous	Medical Data
Multinomial	Discrete (Counts)	Text Classification
Bernoulli	Binary	Spam Detection

Python Implementation of Naive Bayes algorithm

Steps to implement

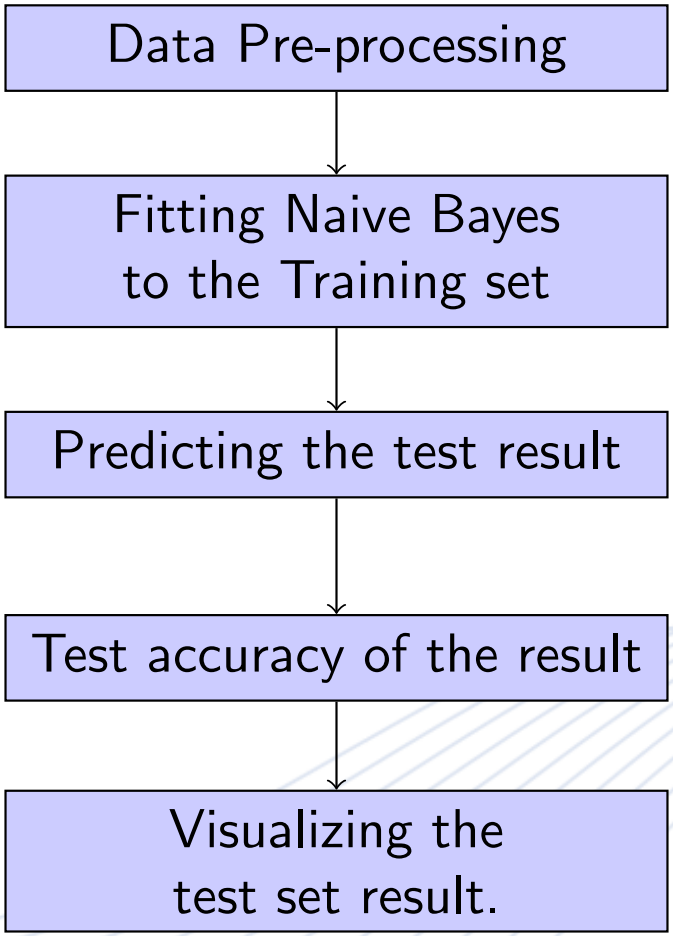

Abbildung: Naïve Bayes Classification Process

```python
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
# Importing the dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, -1].values
# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
# Training the Naive Bayes model on the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
# Predicting the Test set results
y_pred = classifier.predict(X_test)
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
# Visualising the Training set results optional
```

Pros and Cons

Pros and Cons of Naive Bayes Classifier

Pros

- Efficient
- Simple to Implement
- Good for Text Data

Cons

- Assumes Feature Independence
- Zero-Frequency Problem

References

References I

- Chen, Hong u. a. (2021). "Improved naive Bayes classification algorithm for traffic risk management". In: *EURASIP Journal on Advances in Signal Processing* 2021.1, S. 30. DOI: 10.1186/s13634-021-00742-6. URL: https://doi.org/10.1186/s13634-021-00742-6.
- John, George H. und Pat Langley (1995). *An Empirical Study of the Naive Bayes Classifier*. Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence (UAI'95), S. 338–345.
- Khalilpour Darzi, Mohammad Rasoul, Majid Khedmati und Seyed Taghi Akhavan Niaki (2021). *Correlation-Augmented Naïve Bayes (CAN) Algorithm: A Novel Bayesian Method Adjusted for Direct Marketing*. Bd. 35. 15. Applied Artificial Intelligence, Taylor & Francis, S. 2013–2036. DOI: 10.1080/08839514.2021.1997226. URL: https://doi.org/10.1080/08839514.2021.1997226.

References II

- Lewis, David D (1998). "Naive (Bayes) at Forty: The Independence Assumption in Information Retrieval". In: *European Conference on Machine Learning*. Springer, S. 4–15.
- McCallum, Andrew, Kamal Nigam u. a. (1998). "A comparison of event models for naive bayes text classification". In: *AAAI-98 Workshop on Learning for Text Categorization*. Bd. 752. 1. Madison, WI, S. 41–48.
- Peretz, Or, Michal Koren und Oded Koren (2024). *Naive Bayes classifier – An ensemble procedure for recall and precision enrichment*. Bd. 136. Engineering Applications of Artificial Intelligence, S. 108972. DOI: 10.1016/j.engappai.2024.108972. URL: https://www.sciencedirect.com/science/article/pii/S0952197624011308.

References III

Webb, Geoffrey I. (2010). "Naïve Bayes". In: *Encyclopedia of Machine Learning*. Hrsg. von Claude Sammut und Geoffrey I. Webb. Boston, MA: Springer US, S. 713–714. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_576. URL: https://doi.org/10.1007/978-0-387-30164-8_576.

Wolfson, J. u. a. (2015). "A Naive Bayes machine learning approach to risk prediction using censored, time-to-event data". In: *Statistics in Medicine* 34.21, S. 2941–2957. DOI: 10.1002/sim.6526. URL: https://doi.org/10.1002/sim.6526.